8) GroupBy Advanced Concepts

A) cube() B) rollup

C) grouping sets

D) approx_count_distinct()

GroupBy Advanced concepts:

A) cube() :

• # Multi Dimensional Aggregation

↳ cube() generates all possible combinations of grapings for given columns

region	product	sales
East	A	100
East	B	200
West	A	150

→ df.groupBy ("region", "product").sum("sales")

East	A	100
East	B	200
West	A	150

← o/p

with cube():

```
df.cube('region','product').sum('sales')
```

o/p:

East	A	100	
East	B	200	
West	A	150	
East	null	300	← subtotal by region
West	null	150	
null	A	250	← subtotal by product
null	B	200	
null	null	450	← grand total

That's 2^n combinations

Internal Behaviour:

spark:

> Expands dataset logically
> Performs multiple grouping sets
> Causes heavy shuffle
> Generates many aggregation paths

Interview Q's on cube() :

① How many grouping combinations does cube generate?
② Why does cube increase shuffle cost?
③ Why do null values appear?
④ When would you use cube?
⑤ What is the performance complexity with n columns?

---

① If you cube 3 columns, how many grouping combinations?
② Why is cube expensive for 5 columns?
③ Does cube cause shuffle? why?
④ What does null mean in cube o/p?
⑤ Would you use cube in real-time streaming pipeline?

B) rollup():

Hierarchial Aggregation

↳ rollup() creates hierarchical subtotals based on column order.

↳ It does NOT generate all combinations like cube.

↳ It generates:

- Full grouping
- Drop last column
- Drop last two columns
- Grand total

```
df.rollup('region', 'product').sum('sales')
```

o/p:

East	A	100
East	B	200
West	A	150
east	null	300 ← subtotal per region
west	null	150
null	null	450 ← grand total

we don't have

null A 250

null B 200 in the o/p.

~~How rollup expands Internally:~~

~~rollup(A,B,C)~~

~~spark generates~~ ⟹ ~~(A,B,C)~~ ~~(A,B)~~ ~~(A)~~ ~~()~~

~~(total = n+1 combinati~~

~~Why rollup is mo~~

Country	State	City	Sales
India	AP	Guntur	100
India	AP	vizag	200
India	TN	chennai	300
USA	NewYork	xxx	400

Spark Calculates:

only n+1 Combinations

1. Country + State + City (A+B+C)
2. Country + State (A+B)
3. Country (A)
4. Grand Total ( )

O/P ↓

India	AP	Guntur	100
India	AP	vizag	200
India	TN	chennai	300
USA	NY	xxx	400
India	AP	null	300
India	TN	null	300
USA	NY	null	400
India	null	null	800
USA	null	null	400
null	null	null	1000

Interview Q's:

1. Diff b/w cube & rollup?
2. Why is rollup less expensive?
3. Does rollup generate all grouping combinations?
4. How many groupings for 4 columns in rollup?
5. Why does column order matter?

1. If you rollup 4 columns, how many grouping levels?
2. Why is rollup more suitable for hierarchical reports?
3. Does rollup cause shuffle? Why?
4. Would you use rollup for all possible product-region combinations?
5. Which grows faster? (cube/rollup)

c) grouping sets() :

- #custom Aggregation control

cube() → all combinations

rollup() → hierarchical combinations

Region	Product	Year	Sales
East	A	2023	100
East	B	2023	200
west	A	2023	150

If we want only;
region + product
region only
year only
(Not everything)

Sql

```
Select region, product, year, sum(sales)
from sales_table
Group By Grouping sets (
        (region, product),
        (region),
        (year)
)
```

Pyspark:

```
df.groupByExpr('region', 'product', 'year')\
    .agg(....)
# typically SQL used for grouping sets
```

Interview Qs:

1. Diff b/w rollup & grouping sets?
2. When would grouping sets be preferred?
3. Does grouping sets cause shuffle?
4. Is grouping sets more efficient than cube?
5. Can grouping sets produce grand total?

D) approx_count_distinct()

Fast approximate counting
This is performance optimization function

Ex:

```
df.select("user_id").distinct().count()
df.groupBy('region').agg(countDistinct('user_id'))
```

↳ Both are good for small data;
↳ for large data → very expensive.

What approx_count_distinct() Does?

↳ Instead of exact counting
↳ It uses a probabilistic algorithm called (HyperLogLog)

↳ It uses hashing,
stores compressed representation,
estimates unique count,
very memory efficient
very fast

Example:

```
df.groupBy('region')\
  .agg(approx_count_distinct('user_id'))
```

(This gives approximate distinct count per region)

How Accurate is approx_count_distinct()?

(Default relative error ≈ 5%)

↓

we can control it;

↓

approx_count_distinct('user_id', 0.01)

relative error

lower error → more memory → slightly slower

Why its much faster?

Exact countDistinct:

1. Needs full deduplication
2. keeps all unique values
3. heavy shuffle

approx_count_distinct:

1. keeps small hash sketch
2. No need to store all unique values
3. Much lower memory usage.

Interview Q's:

1. why is countDistinct expensive?
2. what algorithm does approx_count_distinct use?
3. Can you control its accuracy?
4. would you use it for payment calculation?
5. Why is it more memory efficient?